AI WORKSPACE · AGENT SHOWCASE

sentry-issue-investigator

An AI agent that investigates our production errors —

and has to prove every answer it gives.

Tools · Location & API modules | Read-only by construction | Every write needs a human "yes"

WHY BUILD THIS

Production errors are cheap to create and expensive to understand

📈 The backlog never sleeps

Sentry collects thousands of events across our modules. Nobody has time to look at all of them, so triage happens by gut feel — or not at all.

⏱️ Each investigation costs hours

Stack trace → code → production logs → maybe the database. It's skilled, repetitive detective work that pulls an engineer out of sprint work every time.

🎭 The worst outcome isn't "no answer"

It's a **confident wrong diagnosis**. A plausible story gets a ticket, the ticket gets a fix, the fix closes — and the system is still broken.

This agent automates the detective work — with rules specifically designed to make the third outcome impossible.

WHAT IT DOES

From an error alert to an evidence-backed diagnosis

1

Reads the error

Pulls the issue from Sentry: stack trace, frequency, how many users, when it started.

2

Reads the actual code

Follows the trace into our repository and reads the real function — not a guess about it.

3

Checks production logs

Tests its hypothesis against what production actually recorded (Elasticsearch).

4

Checks production data

Optionally inspects the database — through a read-only gate, only when enabled.

5

Delivers a verdict

A written diagnosis where every causal claim carries evidence — then, if we agree: ticket → fix plan → PR.

It never writes application code itself. Implementation is handed to a separate agent, and only after an explicit go-ahead.

THE GOLDEN RULE

It is not allowed to tell a plausible story

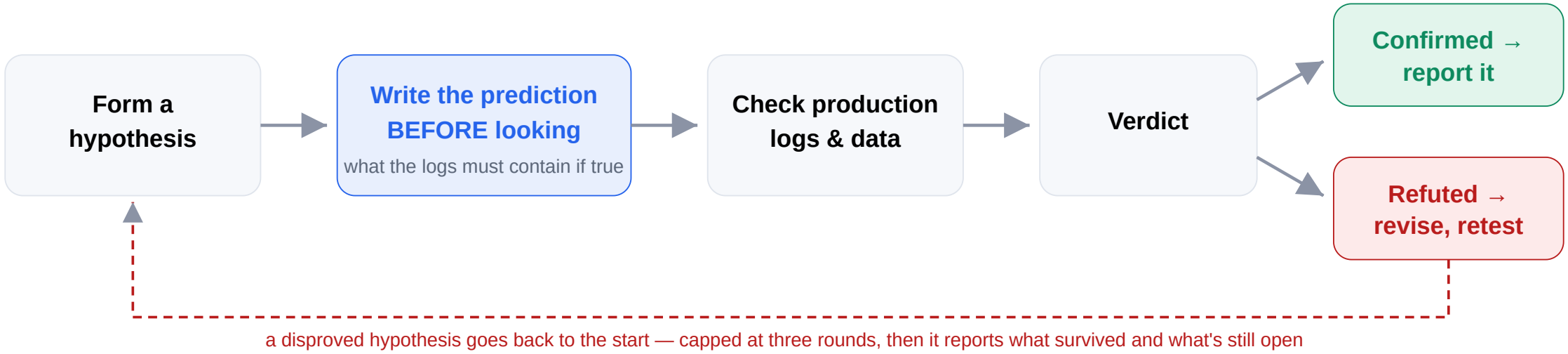
Every link in the causal chain is a **claim**, and every claim must end with one of four verdicts:

VERDICT	PLAIN MEANING
OBSERVED	A tool result shows it directly.
CONFIRMED	It predicted what production would contain, went and looked, and found it.
REFUTED	It predicted, looked properly — and the prediction failed. The hypothesis is wrong; revise it.
UNTESTABLE	Nothing in production can answer it today → it must propose the logging that would.

> **"Sounds right" is not a verdict. The final summary may never sound more confident than the evidence table beneath it.**

HOW AN INVESTIGATION RUNS

It tries to prove itself wrong



Writing the prediction *first* is the whole trick — it stops a query result from being re-interpreted afterwards to fit the story. The same discipline scientists use, applied to log searches.

THE SUBTLE FAILURE IT REFUSES TO MAKE

"No logs found" is not an answer

An empty search result can mean four completely different things. Before it's allowed to say *"this didn't happen"*, three sanity checks must pass:

1 · Is the data still there?

Production logs are kept ~31 days. If the event is older, the evidence *existed and expired* — that's "re-check on recurrence", not "didn't happen".

2 · Did that log ship at all?

If the whole log file is empty for that server and hour, the log isn't reaching our search system — an observability gap worth its own ticket.

3 · Can my search even match?

The search pattern must be shown to match *somewhere*. If it can't, the query is broken — not the hypothesis.

A disconnected VPN and a genuinely quiet system look identical in a results pane: both empty. These checks are how the agent never confuses one for the other.

RUNS BY ITSELF

A triage digest every weekday morning, 08:12

What it surfaces first

Raw frequency buries what matters. Before the big table, it flags:

- 🆕 **Brand-new** — appeared in the last 24h (which deploy introduced it?)
- 🔁 **Came back** — was fixed before; the strongest signal there is
- 📈 **Spiking** — rising fast, even from a small base

Every issue gets a decision — none skipped

ACT NOW	Worth fixing this sprint or next — **only these become Jira tickets**
SNOOZE	Real, not urgent — always with a **numeric threshold** that brings it back ("revisit above 500 events/week")
IGNORE	Safe to leave — with the reason written down

Tickets nobody will pick up cost the board's signal-to-noise and buy nothing — so they don't get filed.

The scheduled run is **look-and-report only** — it cannot file, resolve or snooze anything. And if the environment is unhealthy it says "SKIPPED — not an all-clear" rather than faking a quiet morning.

WHEN THE EVIDENCE RUNS OUT

"I don't know" comes with a plan to make it knowable

What it will not do

Guess a root cause to look decisive. A fix shipped against a guess produces **a closed ticket and a still-broken system** — worse than an open ticket.

What it does instead

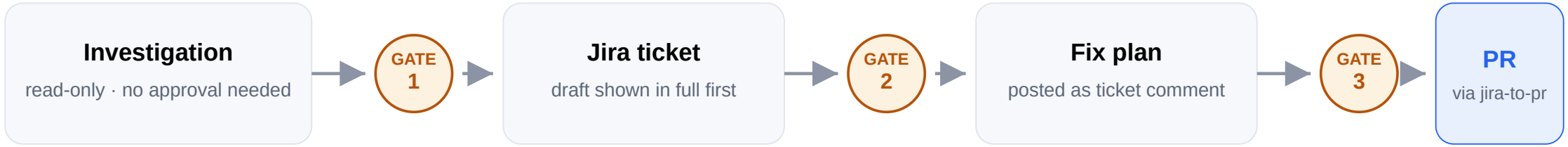
Proposes **exactly what to log** — which file, which line, what to record — so the *next* occurrence answers the question definitively.

Each proposal comes with the search it would enable (written in advance), the expected data volume, a privacy check — and **how long until it pays off**, calculated from the real error rate: "at ~12 events/day, one day after deploy is enough."

That number lets us judge whether instrumenting is worth it *before* agreeing to it.

HUMAN IN THE LOOP

Three gates — never chained on a single "yes"



approving a ticket ≠ approving a plan ≠ approving implementation — each gate is its own question, in its own moment

Duplicates blocked

Before filing anything it searches Jira for an existing ticket on that error — and stops if one exists.

Regressions get named

If the existing ticket is *closed*, that's a returned bug — a previous fix didn't hold. It says so prominently instead of quietly filing a duplicate.

SAFE BY CONSTRUCTION

Read-only isn't a promise — it's how it's built

SURFACE	GUARANTEE
Application code	Never edits it. Implementation goes through a separate agent, behind Gate 3.
Production database	One sanctioned path: a SELECT-only query runner with a parser guard, a read-only transaction, and rollback-on-exit. The guard survived a **17-attempt bypass test** — and its one honest limitation is documented, with a server-side read-only account closing it.
Production logs	Search tools only — the delete/update tools that exist on that server are never called.
Jira & Sentry	Every write shown in full and confirmed first; never resolves, assigns or edits anything unasked.
The morning digest	Runs against an explicit tool allowlist with every write tool denied — safety doesn't depend on the AI behaving, it's enforced outside it.

> **The design assumption: an agent you can trust is one that *couldn't* misbehave, not one that promised it wouldn't.**

ARTIFACTS

What actually lands on your desk

🔍 Investigation report

A verdict up top, a claim-by-claim evidence table beneath it, the causal mechanism built only from proven links — and a "what I could not determine" section that stays honest.

🌅 Daily digest + Slack TL;DR

The morning triage file with signals, the ranked table and a 2–3 issue "what deserves attention today" read — TL;DR posted to Slack.

🎫 Tickets that fit the team

Filed in the right project, right issue type, right priority — following the conventions derived from what the team actually does, with the Sentry link, quantified impact and acceptance criteria.

🔧 Instrumentation proposals

When the cause can't be proven: the exact logging change that would prove it, and the time-to-answer that says whether it's worth it.

Everything is written to the workspace as reviewable files — the chat summary is never the only record.

USING IT

You talk to it in plain language

```
› what's breaking in Tools right now?
› list the top sentry issues in the location module
› investigate TOOLS-BACKEND-B77
› find the production logs for this error
› what would we need to log to know?
› create a Jira ticket for this  ← Gate 1
› add the fix plan to the ticket  ← Gate 2
› implement it  ← Gate 3 → branch, commits, PR
```

Setup is three internal integrations (Sentry, Elasticsearch, Jira) plus the VPN — documented step-by-step in the agent's own README, including the failure modes it already knows about.

WHERE THIS GOES NEXT

The roadmap the agent wrote for itself

- **One join key everywhere.** Its most-proposed improvement: tag every error with the request ID and every log line with the error ID — turning "these two events happened within 2 minutes of each other" into an exact match. Retires its own biggest weakness.
- **More structured logs.** Where services already log structured data, correlation is a real join today — extend that to the rest of Tools.
- **Wider module coverage.** Scoped to Location & API today; the method transfers to any module — and to other teams' boards.
- **The pattern generalises.** Claim ledgers, controls before negatives, and human gates aren't Sentry-specific — they're a template for any AI agent we want to trust with production questions.

> **The goal was never "an AI that answers faster." It's an AI whose answers you can check — and whose silence you can trust.**